ELI — Operations

ELI v2.5.10

October 2026

Contents

Blueprint — Operational Realities **1**
1. Startup / cold-start latency . . . 1
2. Resource contention (daemons vs interactive) . . . 1
3. Memory limits & eviction (two distinct policies) . . . 2
4. Failure recovery — degradation, not rollback . . . 2
5. Testing strategy (two layers + the gaps) . . . 2
6. Recommended cheap, high-value fixes (grounded in §1–§2) . . . 3
Update — 2026-06-09 (scheduled-task dedup; clean shutdown; STT VRAM) . . . 3

Blueprint — Operational Realities

What the structural diagrams (`architecture.md`, `diagrams.md`, `architecture_ascii.md`) **don't** show: cold-start, concurrency/contention, memory eviction, failure recovery, and the testing strategy. Grounded in the code + runtime logs. Estimates are marked *(est.)*; everything else is read from source or measured timings.

---

1. Startup / cold-start latency

No wall-clock delta is printed at boot (a `window._debug_boot_ts = time.time()` hook exists in `gui/eli_pro_audio_gui_v2_0.py` but nothing measures against it), so the number below is reasoned from the boot sequence, not measured.

Cost breakdown (boot order): | Component | Cost | Notes | |—|—|—| | **GGUF model load** | **dominant** | 4.36 GB (7B) / **15.6 GB (24B)** off disk + GPU offload | | **Vision (Moondream) RESIDENT** | medium | loaded at boot, co-resident; **caps ctx 28672→18432** and holds VRAM even if unused | | faster-whisper `small.en` | low–med | CPU int8 | | daemons + 230-cap manifest + persona_updater | low | DB/CPU | | **embedder (nomic) + FAISS** | **lazy** | load on **first message**, not boot → turn-1 latency bump |

Estimate *(est.)*: ~15–40 s warm-cache on the 7B (dominated by model + resident-vision load); materially worse cold and on the 24B (15.6 GB).

Cheap wins: (a) make vision **lazy** (load on first vision use, not boot) — cuts cold start *and* frees VRAM / restores ctx; (b) instrument `_debug_boot_ts` to print per-stage deltas so this stops being an estimate.

---

2. Resource contention (daemons vs interactive)

One shared model behind one `RLock` (`engine.py::_gguf_lock`) — all inference serializes through it.

Verified mitigation: model-using background work uses **non-blocking acquire + defer**. Reflection: `self._gguf_lock.acquire(blocking=False)`, 4 retries, then `"Reflection deferred (GGUF busy)"` and backs off → **yields to interactive turns**. Correct pattern, and real.

Risks (honest): - Reflection demonstrably defers; **news_synthesis / self_improvement / proactive_daemon also call the model** and should all route through the same defer guard — verify each does (a direct blocking `gguf_inference` call would stall an interactive turn). - On the **24B-CPU a turn is minutes**, so a deferred daemon waits minutes for a window, and any non-deferred model call would block your reply for minutes. - **Non-model contention** (cheap but ever-present): `persona_updater` writes the DB **every turn**; the proactive daemon scans the DB; **ambient vision runs the vision model on CPU**; the embedder is a *separate* CPU model — so vector search + ambient vision can hit CPU simultaneously.

---

3. Memory limits & eviction (two distinct policies)

(a) Working-memory pins — `cognition/working_memory.py`: - `MAX_PINS = 20` hard cap; importance-weighted. - `MAX_AGE_TURNS = 40` — `_evict_stale()` drops facts unreferenced for 40 turns. - `_evict_one()` drops lowest-importance-oldest at cap. - Persisted `ORDER BY importance DESC LIMIT 20`. → a principled **importance/age LRU**, bounded.

(b) Context-window fill (`n_ctx`) — a *different* mechanism, and the one that actually bites under pressure: - `engine.py::_build_enhanced_system` budget-trims the **persona**. - `_get_chat_response` **recency-truncates `memory_context`** (keeps the latest, cuts the head) to fit `n_ctx`. - The 20 small pins almost always fit, so **retrieved memory is what gets evicted under pressure, by recency** — not the pins.

(c) Long-term store — SQLite `memories` / `conversation_turns` are **unbounded on disk**; they never fill the window directly. Retrieval (FTS5/FAISS/rerank top-k) gates what enters context. The real "limit" is **retrieval selection + recency truncation**, not a store cap.

---

4. Failure recovery — degradation, not rollback

Grounding escalation (`runtime/grounding_escalation.py`): each tier is wrapped in `try/except`; a failing tier **falls through to the next** (web → local → hedge); the engine hook is itself wrapped (`"grounding escalation skipped"`) so total failure **degrades to the normal CHAT answer**. **No rollback — and none needed — because the tiers are side-effect-free reads** (web search reads; local re-dispatch is read-only). Nothing mutates, so there's nothing to undo.

System-wide: failure handling is **graceful degradation** throughout (broker → direct GGUF → failed-executor surface; failures logged to the `failures` table).

Gap (honest): multi-step write actions (CREATE_FILE, memory writes, a multi-action plan) are individually atomic but **not transactional as a group** — if step 3 fails, steps 1–2 are not rolled back. Rare on the current action set, but real.

---

5. Testing strategy (two layers + the gaps)

Layer 1 — code correctness: 544 pytest files, 12,594 tests collected, all passing on the release run — routing patterns, contracts, executor gates, memory, kernel, perception/voice, packaging. About 14 minutes for the full run; run on every change.

Layer 2 — behavioural eval: `tools/eval/` (see `tools/eval/README.md`) drives the **real pipeline** and asserts on telemetry (action / grounding / response_mode / latency) + text. Cases seeded from fixed bug-logs as permanent regression guards.

Gaps that should exist for a system this complex and don't yet: 1. **No CI** — suites run by manual discipline, not automatically. 2. **No coverage measurement** — hence no exact dead/dormant-code number (a `process()`-under-coverage profiler over the 95 stored conversations would close this). 3. **Behavioural eval is new and small** (~13 cases) — grow to the ~30–50 seeded set and beyond. 4. **No perf/latency regression tracking** and **no multi-model comparison run** standing yet — both enabled by the harness, not yet wired.

---

6. Recommended cheap, high-value fixes (grounded in §1–§2)

Fix	Why	Where
Lazy vision load	cuts cold start + frees VRAM + restores ctx (28672 vs 18432)	`perception/vision.py`, boot in `gui/…MKI.py`
Verify every model-calling daemon uses the non-blocking defer	stops a background job stalling an interactive reply (minutes on 24B)	`runtime/self_improvement.py`, `planning/proactive_daemon.py`, `eli/tools/news/news_synthesis.py` vs `engine.py::_gguf_lock`
Boot-timing instrumentation	turns "cold start" from estimate → measured	`gui/…MKI.py _debug_boot_ts`
Coverage profiler over stored conversations	exact dead/dormant-code map	new `tools/eval/` profiler

Companion docs: `architecture.md` · `architecture_ascii.md` · `diagrams.md` · `tools/eval/README.md` (the measurement layer). "'

Update — 2026-06-09 (scheduled-task dedup; clean shutdown; STT VRAM)

- **Scheduled-task dedup.** Standing nightly jobs (testgen/eval/report) were re-armed on every boot AND on completion with no dedup, accumulating copies (observed: 4× testgen / 3× eval). `schedule_request` now keeps ONE entry per (kind, request) for recurring jobs, and `restore_scheduled_tasks` collapses existing duplicates on boot. With the model-load VRAM interlock (a background worker waits for `gguf_inference.is_loaded()` and skips rather than cold-loading a second model), background jobs can no longer pile up or starve the main model at boot.
- **Clean shutdown (no segfault).** The GUI exited via `sys.exit(app.exec())`, so CPython ran llama.cpp + FAISS C++ destructors at interpreter teardown and segfaulted — AFTER state was already flushed (session summary written), so nothing was lost, but it dumped core. `main()` now runs the shutdown/atexit flush (memory + session summary + explicit model unload) then `os._exit(rc)`, bypassing the destructor pass.
- **STT no longer starves the main model:** faster-whisper is VRAM-aware (CPU on ≤8 GB cards), so the main model reclaims its GPU layers (gpu_layers 11→99). See `perception.md`.